ESPIGA

DOCUMENTO TÉCNICO EJECUTIVO · PROYECTO FINAL 2026

Documentación Técnica del Proyecto Final

Recorrido del código y herramientas

Proyecto Espiga · PCR Amoblamientos **Materia** Proyecto, Diseño e Implementación de Sistemas Computacionales

Integrantes Felipe Araujo · Valentina Autelli · Mía Yorio **Fecha** 12/11/2026

Este documento recorre cómo está construido Espiga, el sistema de gestión de PCR Amoblamientos: qué partes lo forman, dónde corre cada una y qué archivos del repositorio `felooaraujo-ai/Kore` las implementan. Seguimos dos casos reales de punta a punta, desde el botón en la pantalla hasta la fila en la base de datos, y cerramos con las herramientas usadas, incluida la inteligencia artificial. Todas las rutas y nombres coinciden con la rama `main` desplegada en producción.

SECCIÓN 1

Las partes del sistema y cómo se relacionan

El navegador (PC o celular) solo habla con el servidor, siempre por HTTPS y en formato JSON (los PDF y Excel viajan como archivo). El servidor es el único que lee y escribe la base de datos MySQL: el frontend nunca ve credenciales ni hace consultas SQL. Los servicios externos (mail, notificaciones) también se llaman únicamente desde el servidor.

Parte	Dónde corre	Qué hace	Archivos principales
Página web	Navegador del usuario (instalable como app, PWA)	Pantallas, formularios, gráficos; llama a la API con *fetch*	`public/index.html`, `public/seguimiento.html`, `public/sw.js`, `public/manifest.json`
Servidor	Node.js + Express en Railway (espiga-pcr.up.railway.app)	Login, permisos, reglas de negocio, PDFs, tareas programadas	`src/server.js`, `src/routes/*.routes.js`, `src/middleware/auth.js`, `src/jobs/`
Base de datos	MySQL en Railway (red privada)	19 tablas, 6 vistas de saldos/alertas y 2 triggers	`sql/01_schema.sql` … `sql/14_pedido_flete_dias.sql`, `src/config/db.js`, `src/config/migraciones.js`
Integraciones	Servicios de terceros, llamados desde el servidor	Mails (Brevo), notificaciones push, archivos subidos	`src/config/mailer.js`, `src/config/push.js`, `src/config/uploads.js`

SECCIÓN 2

La página web (Frontend)

Es una aplicación de una sola página escrita en HTML, CSS y JavaScript sin frameworks, en el archivo `public/index.html`. Cada pantalla es una vista que se dibuja con funciones `render…()` y se navega con `go(id)`. Toda comunicación pasa por una única función, `apiFetch()`, que agrega el token y traduce los errores del servidor a mensajes legibles; el objeto `Api` agrupa una llamada por operación.

- **Ingreso:** `hacerLogin()` envía usuario y clave a `POST /api/auth/login`, guarda el token JWT y el usuario en memoria (`AUTH`) y carga los datos con `cargarTodo()`. Desde el login se accede a “Olvidé mi clave”.
- **Operación principal:** `renderInicio()` (resumen del día), clientes y pedidos con ítems, fotos y materiales, presupuesto con flete y plazo en días hábiles, producción, proveedores con varias cuentas, insumos y usuarios. Ejemplo: `guardarPago()` → `Api.crearPago()` → `fetch` a `/api/pedidos/:id/pagos`.
- **Reportes y comprobantes:** panel de saldos y estadísticas con Chart.js (CDN cdnjs), descarga de presupuesto, factura y resumen en PDF, y exportación a Excel. El cliente final consulta su pedido en `/seguimiento` con un link propio.
- **Detalles comunes:** diseño adaptable a celular con menú lateral desplazable, tipografía Inter, modo claro/oscuro, avisos tipo *toast*, botones ocultos según el rol (mapa `PUEDE`) y manejo de errores de red (“No se pudo conectar con

el servidor"). El Service Worker `sw.js` solo recibe notificaciones y no guarda cachés viejos.

SECCIÓN 3

El servidor (Backend)

`src/server.js` arma la aplicación Express: habilita CORS y JSON, monta cada archivo de rutas bajo `/api`, sirve la carpeta `public`, responde 404 a rutas inexistentes, corre las migraciones al iniciar y programa las tareas periódicas. Cada archivo de `src/routes/` agrupa un tema:

Rutas / Scripts	Para qué sirve
`/api/auth` · auth.routes.js	Login, "olvidé mi clave" y reseteo con token por mail
`/api/clientes` · clientes, resumen	ABM de clientes, saldo por cliente y resumen de cuenta en PDF
`/api/pedidos` · pedidos, presupuesto, factura	Pedidos, ítems, fotos, estados, flete, entrega, pagos, facturas y PDFs con QR
`/api/proveedores` · `/api/insumos`	Proveedores con cuentas separadas y movimientos; stock de insumos
`/api/panel` · `/api/exportar`	Saldos, alertas, estadísticas, flujo de caja; Excel de clientes, pedidos y pagos
`/api/usuarios` · `/api/perfil` · `/api/admin`	Alta de usuarios y roles, cambio de clave propia, pase a "modo cliente"
`/api/push` · `/api/backups` · `/api/seguimiento`	Suscripción a avisos, descarga de backups y link público del cliente
`src/jobs/` alertas.job.js, backup.job.js	Cada hora revisa entregas próximas; backup diario y envío semanal por mail

- **Quién es quién:** al ingresar, el servidor firma un token JWT (vence a las 8 h) con el id y el rol. `requireAuth` en `src/middleware/auth.js` lo verifica en cada pedido (si falta o venció: 401) y `requireRole(...)` corta con 403 si el rol (administrador, administrativo u operario) no tiene permiso. Los permisos se validan en el servidor; el frontend solo esconde botones.
- **Secretos:** las claves se guardan con **bcrypt** (nunca en texto plano). `JWT_SECRET`, datos de MySQL, la API key de Brevo y las claves VAPID viven en variables de entorno (`.env` local, panel de Railway en producción); `.env` está en `.gitignore` y en GitHub solo hay `.env.example` con valores de ejemplo. Tras 5 intentos fallidos el login se bloquea 5 minutos (`src/config/seguridad.js`, respuesta 429).
- **Validación y cálculos:** cada ruta controla los campos obligatorios (400 si faltan) y todas las consultas usan parámetros `?` de mysql2, lo que evita la inyección SQL. Los saldos no los calcula el navegador ni se guardan: salen de vistas SQL (`v_saldo_pedido` suma flete, resta pagos) y los días hábiles con feriados se calculan en `src/config/diasHabiles.js`.

SECCIÓN 4

La base de datos

Motor **MySQL** alojado en **Railway**, conectado con un pool de mysql2 (`src/config/db.js`). El esquema se crea con los scripts `sql/01_schema.sql` a `sql/14_pedido_flete_dias.sql` (tablas, `02_vistas.sql`, `03_triggers.sql`, datos de prueba en `04_seed.sql`) y `src/config/migraciones.js` aplica los cambios nuevos al arrancar. Tablas principales:

Tabla	Qué guarda	Tabla	Qué guarda
`usuario`	Login, hash de clave (bcrypt), rol, token de recuperación	`proveedor`	Datos del proveedor
`cliente`	Datos de contacto del cliente	`proveedor_cuenta`	Cuentas separadas de cada proveedor
`pedido`	Trabajo, monto, flete, días hábiles, entrega, estado, token de seguimiento	`proveedor_movimiento`	Compras y pagos por cuenta
`pedido_item` · `pedido_foto`	Ítems del presupuesto y fotos del avance	`factura_proveedor`	Facturas recibidas
`pago`	Cobros: fecha, monto, medio, quién lo cargó	`insumo` · `insumo_movimiento`	Stock y entradas/salidas
`material` · `adjunto`	Materiales usados y archivos del pedido	`push_suscripcion` · `push_enviado`	Celulares suscriptos y avisos ya mandados
`factura_cliente`	Facturas emitidas al cliente	`push_seguimiento`	Avisos al cliente final
`pedido_estado_historial`	Cada cambio de estado con fecha y usuario (trazabilidad)		

Vistas: `v_saldo_pedido`, `v_saldo_cliente`, `v_deuda_proveedor`, `v_deuda_proveedor_cuenta`, `v_panel_saldos`, `v_alertas_entrega`. Triggers: `trg_pedido_before_update` y `trg_pago_after_insert` cierran solos un pedido entregado y sin saldo.

SECCIÓN 5

Integraciones externas

- **Correo con Brevo (API HTTP):** mails de recuperación de clave y backups semanales (`src/config/mailer.js`). Se usa la API porque Railway bloquea el puerto SMTP.
- **Notificaciones Web Push (VAPID):** librería web-push (`src/config/push.js`) para avisar entregas al celular del taller y novedades al cliente que sigue su pedido.
- **Documentos:** PDFs de presupuesto (con QR al seguimiento, pdfkit + qrcode), factura y resumen; Excel con exceljs.
- **Hosting y archivos:** Railway despliega automáticamente cada push a `main`; las fotos se guardan con multer en un volumen persistente (`UPLOAD_DIR`).

SECCIÓN 6

El recorrido de punta a punta

Caso A · La secretaria registra un pago desde el celular

01 **Pantalla:** en la ficha del cliente toca “Registrar pago”, completa fecha, monto y medio; `guardarPago()` en `public/index.html` llama a `Api.crearPago()`.

02 **Viaje:** `apiFetch()` envía `POST /api/pedidos/:id/pagos` por HTTPS con JSON y el encabezado `Authorization: Bearer <JWT>`.

03 **Control:** en `src/routes/pedidos.routes.js`, `requireAuth` valida el token y `requireRole('administrador','administrativo')` el rol; si falta fecha, monto o medio responde 400.

04 **Base de datos:** `INSERT INTO pago (pedido_id, usuario_id, fecha, monto, medio) VALUES (?, ?, ?, ?, ?)`; el trigger `trg_pago_after_insert` lee `v_saldo_pedido` y, si el pedido ya estaba entregado y el saldo llegó a 0, lo pasa a “cerrado”.

05 **Respuesta:** el servidor devuelve 201 con el id; la app muestra “Pago de $… registrado ✓”, vuelve a la ficha y el saldo (ahora calculado por la vista) aparece actualizado en la ficha y en el Panel.

Caso B · Aviso automático de entrega próxima al celular de Rodrigo

01 **Disparo:** en `src/server.js` un `setInterval` ejecuta cada hora `revisarYNotificar()` de `src/jobs/alertas.job.js` (si faltan las claves VAPID, se saltea).

02 **Consulta:** lee la vista `v_alertas_entrega`: pedidos en producción cuya fecha de entrega está a 7 días o menos (o ya vencida), con `dias_restantes`.

03 **Sin repetir:** para cada pedido revisa `push_enviado`; si ya se avisó hoy lo saltea. Arma el texto: “Entrega en 3 día(s)”, “Entrega HOY” o “Atrasado”.

04 **Envío:** `webpush.sendNotification()` manda el aviso a cada fila de `push_suscripcion`; si el navegador responde 404/410 borra esa suscripción vencida, y registra el envío en `push_enviado`.

05 **Celular:** `public/sw.js` recibe el evento `push` y muestra la notificación; al tocarla abre `/?pedido=ID` directo en el pedido.

SECCIÓN 7

Herramientas utilizadas

Categoría	Herramientas
Lenguajes	JavaScript (Node.js en el servidor y en el navegador), HTML5, CSS3 y SQL (MySQL).
Backend y Frontend	Express, mysql2, jsonwebtoken, bcrypt, multer, pdfkit, qrcode, exceljs, web-push, nodemailer, dotenv, cors (ver `package.json`); frontend en JavaScript sin frameworks + Chart.js 4.4.1; PWA con `manifest.json` y Service Worker.
Base de datos y Hosting	MySQL en Railway; servidor Node en Railway con volumen persistente y deploy automático; landing y documentación en GitHub Pages.
GitHub y Pruebas (QA)	Repositorio privado `felooaraujo-ai/Kore` (commits a `main` → deploy en Railway). Pruebas: `node --check` sobre cada archivo modificado, revisión de logs de deploy, prueba manual de cada rol (administrador/administrativo/operario) en celular y PC, y verificación de saldos contra las vistas SQL.
Inteligencia Artificial	**Plataforma:** Claude, de Anthropic, usado como Claude Code en la nube desde claude.ai. **Modelos exactos:** Claude Opus 4.8 (`claude-opus-4-8`) y Claude Opus 5.5 (`claude-opus-5-5`). **Tareas asistidas:** escritura y corrección de rutas y vistas SQL, diseño de pantallas, generación de PDFs, documentación y landing. **Validación:** el equipo revisó cada cambio en GitHub antes de darlo por bueno, lo probó en producción con datos reales del taller y con Rodrigo, y no se cargaron credenciales en el chat ni en el repositorio.